

Lambda and Kappa architectures: The Lambda architecture allows for separating your pipeline by including a batch layer to ensure accuracy and a speed layer to provide insights in real-time. While this is a powerful model, it is considerably more complex than other architectures as it requires you to maintain two codebases, is sensitive to coordination points, and has the potential to produce inconsistencies if not managed correctly.

The Kappa architecture uses only stream processing, even when re-processing historical data, so it doesn't have the same complexity as the Lambda architecture. It also improves the usability of your data pipeline by using a tool like Apache Kafka together with Flink, which processes data before storing the result. Then, depending on your use case, you access the result as it is stored, or you can replay historic events to process them just like the original system would.

Real-world example of hybrid pipelines: An example of a hybrid pipeline is a real-time system (Kafka and Flink) that captures customer clicks and order streams for customer personalisation at the point of sales. It uses Apache Spark to analyse complete transaction logs, once a night, in batch, for analysis of sales, returns, and fraudulent transactions. This hybrid architecture or model allows for both real-time business operations and long-term business intelligence.

Modern enterprises combine batch and stream processing using tools like Delta Lake, Apache Beam, and Google Dataflow to create hybrid pipelines. Hybrid pipelines enable organisations to have unified methods of ingesting and processing data resulting in fast and consistent decision-making processes across the organisation, while also maintaining historical accuracy and compliance with regulations. Such models are not compromises. They are strategies to address complex data challenges, while providing scalability and flexibility.

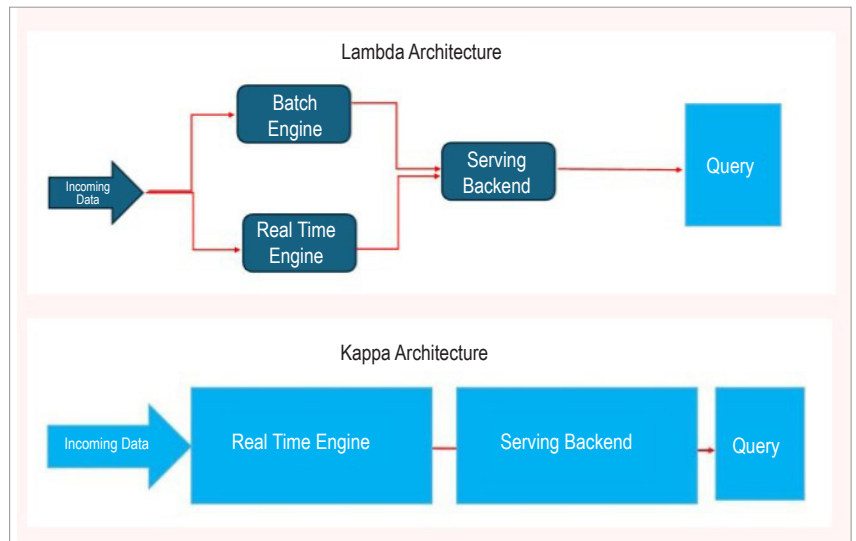


Figure 4: Lambda and Kappa architectures

Open source in action: Case studies

Open source tools are the base of real success stories across many industries, with organisations deploying streaming and batch tools in innovative and impactful ways for everything from logistics to banking and IoT. Let's look at three field examples.

Logistics start-up using Kafka + Flink: A fast-growing logistics company in India had a need to track thousands of deliveries in real time. They onboarded Apache Kafka to ingest GPS data from the delivery vans and Apache Flink to process the stream on-the-fly. Suitable thresholds were set up to detect route deviations, real-time slowdowns in traffic or, even worse, vans idling for long periods. An alert was generated in seconds by the system. This stream-first architecture allowed them to reduce delivery time by 12% and increase customer satisfaction, all with an open source stack.

NiFi and Hive used by a bank: A mid-sized private bank utilised Apache NiFi to automate the secure ingestion of data from hundreds of branches. Data included everything from transaction logs to customer updates. This data was streamed into HDFS. Once there,

the bank used Apache Hive for daily risk assessments, fraud detections, and compliance reporting. The low-code, scalable architecture allowed the bank to eliminate expensive commercial ETL tools and increase efficiency in their regulatory workflows.

IoT use case with Spark Streaming: An energy company outfitted an entire city with smart meters to monitor electricity usage. Each meter was generating data every few seconds. With Spark Structured Streaming the company could identify patterns and analyse data in almost real time for outages, inefficiencies, and surges. Insights were shown on dashboards for engineers and alerts triggered for anomalies so that the company could act on them in a matter of minutes.

The future

As businesses demand quicker decisions and agile infrastructure, the world is starting to transition towards intelligent, unified and dynamic data pipelines. The future is aligned with architectures that support both streaming and batch processing, automate harvesting costs, and auto-enable load based on workloads.

Continued to page...53